LAPORAN PRAKTIKUM

PEMROGRAMAN MOBILE

Pertemuan 5: Penyimpanan Lokal (SharedPreferences dan SQLite)

Nama	:	Muhamad Dika Ramadhan
NIM	:	20240801045
Program Studi	:	Teknik Informatika
Mata Kuliah	:	Pemrograman Mobile
Pertemuan	:	5

1. Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memilih jenis penyimpanan lokal yang tepat: key-value, database, atau berkas.
2. Menyimpan dan membaca pengaturan sederhana dengan shared_preferences.
3. Merancang tabel dan membuat database SQLite dengan sqflite.
4. Mengimplementasikan operasi CRUD (Create, Read, Update, Delete) dan menghubungkannya dengan UI.
5. Memastikan data tetap ada setelah aplikasi ditutup dan dibuka kembali.

2. Landasan Teori

Data di memori (variabel dan State) hilang begitu aplikasi ditutup. Agar data bertahan, aplikasi menyimpannya di penyimpanan perangkat. Hal ini disebut persistence. Pilihan penyimpanan lokal disesuaikan dengan jenis datanya:

Pilihan	Cocok untuk	Paket
Key-value	Pengaturan kecil: tema, nama, status login, angka terakhir.	shared_preferences
Database SQLite	Data terstruktur dan banyak: catatan, transaksi, daftar produk.	sqflite, path
Berkas	Dokumen, gambar, ekspor/impor data.	path_provider, dart:io

CRUD dan SQL

Setiap operasi CRUD memiliki padanan perintah SQL dan method pada sqflite:

CRUD	SQL	Method sqflite
Create	INSERT	db.insert()
Read	SELECT	db.query()
Update	UPDATE	db.update()
Delete	DELETE	db.delete()

Aman dari SQL injection. Query tidak boleh disusun dengan menyambung teks dari pengguna. Gunakan tanda tanya sebagai placeholder lewat where: 'id = ?' dan whereArgs: [id].

3. Alat dan Bahan

- Flutter SDK, editor, dan emulator/perangkat Android (sqflite paling stabil di Android/iOS)
- Proyek baru: flutter create praktikum_5
- Paket shared_preferences, sqflite, dan path (flutter pub add)
- Koneksi internet untuk mengunduh paket (setelah itu tidak diperlukan)

4. Langkah dan Hasil Praktikum

Bagian A. SharedPreferences (Pengaturan Sederhana)

Paket dipasang terlebih dahulu, lalu dibuat aplikasi yang menyimpan nama, pilihan mode gelap, dan hitungan berapa kali aplikasi dibuka.

```
flutter pub add shared_preferences
```

```
class _PengaturanPageState extends State<PengaturanPage> {
  final _controller = TextEditingController();
  bool _gelap = false;
  int _bukaKe = 0;

  @override
  void initState() {
    super.initState();
    _muat();
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  Future<void> _muat() async {
```

```
    final prefs = await SharedPreferences.getInstance();
    if (!mounted) return;
    setState(() {
      _controller.text = prefs.getString('nama') ?? '';
      _gelap = prefs.getBool('gelap') ?? false;
      _bukaKe = (prefs.getInt('bukaKe') ?? 0) + 1;
    });
    await prefs.setInt('bukaKe', _bukaKe);
  }

  Future<void> _simpanNama() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString('nama', _controller.text.trim());
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Nama disimpan')),
    );
  }

  Future<void> _ubahTema(bool nilai) async {
    setState(() => _gelap = nilai);
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool('gelap', nilai);
  }

  @override
  Widget build(BuildContext context) {
    return Theme(
      data: ThemeData(
        brightness: _gelap ? Brightness.dark : Brightness.light,
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ),
      child: Scaffold(
        appBar: AppBar(title: const Text('Pengaturan')),
        body: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            children: [
              TextField(
                controller: _controller,
                decoration: const InputDecoration(
                  labelText: 'Nama',
                  border: OutlineInputBorder(),
                ),
              ),
              const SizedBox(height: 12),
              ElevatedButton(
                onPressed: _simpanNama,
                child: const Text('Simpan nama'),
              ),
              SwitchListTile(
                title: const Text('Mode gelap'),
                value: _gelap,
```

```
                onChanged: _ubahTema,
              ),
              const SizedBox(height: 12),
              Text('Aplikasi dibuka ke-$_bukaKe kali'),
            ],
          ),
        ),
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
import 'package:shared_preferences/shared_preferences.dart'	Mengimpor paket penyimpanan key-value.
SharedPreferences.getInstance()	Membuka akses ke penyimpanan key-value. Bersifat asinkron sehingga harus di-await.
Future<void> _muat() async	Method asinkron untuk membaca data tersimpan. Dipanggil di initState() sehingga pengaturan dimuat saat halaman dibuka.
prefs.getString('nama') ?? ''	Membaca String dengan kunci "nama". Operator ?? memberi nilai bawaan (string kosong) bila belum pernah disimpan, sehingga tidak null.
prefs.getBool('gelap') ?? false	Membaca nilai true/false untuk mode gelap; bawaannya false.
(prefs.getInt('bukaKe') ?? 0) + 1	Membaca hitungan sebelumnya lalu menambah 1 karena aplikasi baru saja dibuka.
prefs.setInt / setString / setBool	Menyimpan nilai ke penyimpanan dengan kunci tertentu. Data tetap ada walau aplikasi ditutup.
if (!mounted) return	Memastikan halaman masih ada di layar setelah await sebelum memakai setState atau context, agar tidak terjadi galat (peringatan use_build_context_synchronously).
_controller.text = ...	Mengisi kolom nama dengan nilai yang tersimpan.
_simpanNama()	Menyimpan teks nama lalu menampilkan SnackBar "Nama disimpan".
_ubahTema(bool nilai)	Mengubah tampilan seketika lewat setState, lalu menyimpan pilihan ke penyimpanan.
SwitchListTile	Baris berisi saklar (switch) untuk mode gelap; onChanged memanggil _ubahTema.

Bagian Kode	Penjelasan
Theme(data: ThemeData(brightness: ...))	Menerapkan tema terang atau gelap pada halaman ini sesuai nilai _gelap.
dispose()	Melepas TextEditingController saat halaman ditutup.

Agar Bagian A dan Bagian B–D dapat dibuka dari satu aplikasi, ditambahkan halaman induk dengan navigasi bawah:

```
// Halaman induk dengan navigasi bawah (Bagian A dan B-D
// digabung dalam satu aplikasi)
class HomePage extends StatefulWidget {
  const HomePage({super.key});

  @override
  State<HomePage> createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {
  int _index = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: IndexedStack(
        index: _index,
        children: const [PengaturanPage(), CatatanPage()],
      ),
      bottomNavigationBar: NavigationBar(
        selectedIndex: _index,
        onDestinationSelected: (i) => setState(() => _index = i),
        destinations: const [
          NavigationDestination(
              icon: Icon(Icons.settings),
              label: 'Pengaturan (A)'),
          NavigationDestination(
              icon: Icon(Icons.sticky_note_2),
              label: 'Catatan SQLite (B-D)'),
        ],
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
NavigationBar + NavigationDestination	Bilah navigasi bawah berisi dua tujuan: Pengaturan (A) dan Catatan SQLite (B–D).
selectedIndex / onDestinationSelected	Menentukan tab yang aktif dan mengubahnya lewat setState saat tab diketuk.

Bagian Kode	Penjelasan
IndexedStack	Menampilkan satu halaman sesuai index namun menjaga halaman lain tetap hidup, sehingga state dan data tidak hilang saat berpindah tab.

Bagian B. Menyiapkan SQLite

Paket sqflite dan path dipasang, lalu dibuat model Catatan dan kelas pembantu database DbHelper.

```
flutter pub add sqflite path
```

```
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

class Catatan {
  final int? id;
  final String judul;
  final String isi;
  const Catatan({this.id, required this.judul, required this.isi});

  Map<String, Object?> toMap() =>
      {'id': id, 'judul': judul, 'isi': isi};

  factory Catatan.fromMap(Map<String, Object?> m) => Catatan(
        id: m['id'] as int,
        judul: m['judul'] as String,
        isi: m['isi'] as String,
      );
}

class DbHelper {
  static Database? _db;

  static Future<Database> get database async {
    if (_db != null) return _db!;
    final path = p.join(await getDatabasesPath(), 'catatan.db');
    _db = await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) {
        return db.execute(
          'CREATE TABLE catatan('
          'id INTEGER PRIMARY KEY AUTOINCREMENT, '
          'judul TEXT NOT NULL, '
          'isi TEXT NOT NULL)',
        );
      },
    );
    return _db!;
  }
```

```
  static Future<int> tambah(Catatan c) async {
    final db = await database;
    return db.insert('catatan', c.toMap());
  }

  static Future<List<Catatan>> semua() async {
    final db = await database;
    final rows = await db.query('catatan', orderBy: 'id DESC');
    return rows.map(Catatan.fromMap).toList();
  }

  static Future<int> ubah(Catatan c) async {
    final db = await database;
    return db.update('catatan', c.toMap(),
        where: 'id = ?', whereArgs: [c.id]);
  }

  static Future<int> hapus(int id) async {
    final db = await database;
    return db.delete('catatan', where: 'id = ?', whereArgs: [id]);
  }
}
```

Bagian Kode	Penjelasan
import 'package:path/path.dart' as p	Paket path untuk menyusun lokasi file dengan benar di berbagai platform (p.join).
class Catatan	Model satu baris tabel catatan: id, judul, dan isi.
final int? id	id bertipe nullable. Bernilai null saat catatan baru dibuat, lalu diisi otomatis oleh SQLite (AUTOINCREMENT).
toMap()	Mengubah objek menjadi Map agar bisa dimasukkan ke database (objek → baris).
Catatan.fromMap(...)	Mengubah Map hasil query menjadi objek Catatan (baris → objek).
static Database? _db	Menyimpan satu koneksi database yang dipakai bersama (pola singleton).
static Future<Database> get database async	Getter yang membuka database sekali saja: bila _db sudah ada langsung dikembalikan, bila belum dibuka lewat openDatabase.
getDatabasesPath()	Mengambil folder standar penyimpanan database di perangkat.
p.join(..., 'catatan.db')	Menggabungkan folder dan nama file menjadi path lengkap database.

Bagian Kode	Penjelasan
openDatabase(path, version, onCreate)	Membuka database; bila file belum ada, dibuat dulu dan onCreate dijalankan. version dipakai untuk pengelolaan perubahan skema.
db.execute('CREATE TABLE ...')	Menjalankan perintah SQL pembuat tabel catatan.
id INTEGER PRIMARY KEY AUTOINCREMENT	Kolom id berupa angka, unik, dan bertambah otomatis.
judul TEXT NOT NULL	Kolom teks yang wajib diisi (tidak boleh kosong).
db.insert('catatan', c.toMap())	Create: menyisipkan satu baris baru; mengembalikan id baris tersebut.
db.query('catatan', orderBy: 'id DESC')	Read: mengambil semua baris, diurutkan dari id terbesar (catatan terbaru di atas).
rows.map(Catatan.fromMap).toList()	Mengubah setiap baris hasil query menjadi objek Catatan.
db.update(..., where: 'id = ?', whereArgs: [c.id])	Update: mengubah baris yang id-nya sama. Tanda ? adalah placeholder yang diisi dari whereArgs sehingga aman dari SQL injection.
db.delete(..., where: 'id = ?', whereArgs: [id])	Delete: menghapus baris dengan id tertentu.

Bagian C. Halaman Daftar Catatan (Read dan Delete)

Properti home pada MyApp diubah menjadi halaman yang memuat CatatanPage, lalu halaman daftar ditambahkan:

```
class _CatatanPageState extends State<CatatanPage> {
  late Future<List<Catatan>> _future;

  @override
  void initState() {
    super.initState();
    _future = DbHelper.semua();
  }

  void _muat() {
    setState(() {
      _future = DbHelper.semua();
    });
  }

  Future<void> _buka([Catatan? catatan]) async {
    await Navigator.push(
      context,
      MaterialPageRoute(
          builder: (_) => FormCatatanPage(catatan: catatan)),
    );
```

```
    if (!mounted) return;
    _muat();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Catatan Saya')),
      body: FutureBuilder<List<Catatan>>(
        future: _future,
        builder: (context, snapshot) {
          if (snapshot.connectionState ==
              ConnectionState.waiting) {
            return const Center(
                child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return Center(
                child: Text('Galat: ${snapshot.error}'));
          }
          final data = snapshot.data!;
          if (data.isEmpty) {
            return const Center(
                child: Text('Belum ada catatan'));
          }
          return ListView.builder(
            itemCount: data.length,
            itemBuilder: (context, i) {
              final c = data[i];
              return ListTile(
                title: Text(c.judul),
                subtitle: Text(c.isi,
                    maxLines: 2,
                    overflow: TextOverflow.ellipsis),
                onTap: () => _buka(c),
                trailing: IconButton(
                  icon: const Icon(Icons.delete),
                  onPressed: () async {
                    await DbHelper.hapus(c.id!);
                    _muat();
                  },
                ),
              );
            },
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => _buka(),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
_future = DbHelper.semua()	Memulai pengambilan data dari database sekali di initState().
_muat()	Membuat Future baru di dalam setState sehingga FutureBuilder memuat ulang daftar. Dipanggil setelah tambah, ubah, atau hapus.
Future<void> _buka([Catatan? catatan])	Membuka halaman form. Tanda kurung siku berarti parameter opsional: kosong untuk catatan baru, terisi untuk mengubah catatan.
await Navigator.push(...)	Menunggu sampai halaman form ditutup, lalu baris berikutnya dijalankan.
if (!mounted) return; _muat();	Setelah kembali dari form, daftar dimuat ulang agar menampilkan data terbaru.
FutureBuilder<List<Catatan>>	Menampilkan tiga status: loading (CircularProgressIndicator), galat (teks galat), dan data.
data.isEmpty	Bila daftar kosong tampil teks "Belum ada catatan".
ListView.builder + ListTile	Menampilkan setiap catatan dengan judul dan isi.
maxLines: 2, overflow: TextOverflow.ellipsis	Isi catatan dibatasi dua baris dan dipotong dengan "..." bila panjang.
onTap: () => _buka(c)	Mengetuk catatan membuka form dalam mode ubah.
IconButton(Icons.delete)	Delete: menghapus catatan dengan DbHelper.hapus(c.id!) lalu memuat ulang daftar.
FloatingActionButton → _buka()	Tombol + membuka form kosong untuk catatan baru.

Bagian D. Form Tambah dan Ubah Catatan (Create dan Update)

Satu halaman form dipakai untuk dua keperluan: bila catatan kosong berarti tambah, bila terisi berarti ubah.

```
class FormCatatanPage extends StatefulWidget {
  final Catatan? catatan;
  const FormCatatanPage({super.key, this.catatan});

  @override
  State<FormCatatanPage> createState() => _FormCatatanPageState();
}

class _FormCatatanPageState extends State<FormCatatanPage> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _judul;
  late final TextEditingController _isi;
```

```
@override
void initState() {
  super.initState();
  _judul = TextEditingController(
      text: widget.catatan?.judul ?? '');
  _isi = TextEditingController(
      text: widget.catatan?.isi ?? '');
}

@override
void dispose() {
  _judul.dispose();
  _isi.dispose();
  super.dispose();
}

Future<void> _simpan() async {
  if (!_formKey.currentState!.validate()) return;
  final c = Catatan(
    id: widget.catatan?.id,
    judul: _judul.text.trim(),
    isi: _isi.text.trim(),
  );
  if (widget.catatan == null) {
    await DbHelper.tambah(c);
  } else {
    await DbHelper.ubah(c);
  }
  if (!mounted) return;
  Navigator.pop(context);
}

@override
Widget build(BuildContext context) {
  final baru = widget.catatan == null;
  return Scaffold(
    appBar: AppBar(
        title: Text(baru ? 'Catatan Baru' : 'Ubah Catatan')),
    body: Form(
      key: _formKey,
      child: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          TextFormField(
            controller: _judul,
            decoration: const InputDecoration(
              labelText: 'Judul',
              border: OutlineInputBorder(),
            ),
            validator: (v) => (v == null || v.trim().isEmpty)
                ? 'Judul wajib diisi' : null,
          ),
          const SizedBox(height: 12),
```

```
            TextFormField(
              controller: _isi,
              maxLines: 6,
              decoration: const InputDecoration(
                labelText: 'Isi catatan',
                border: OutlineInputBorder(),
              ),
              validator: (v) => (v == null || v.trim().isEmpty)
                  ? 'Isi wajib diisi' : null,
            ),
            const SizedBox(height: 16),
            ElevatedButton(onPressed: _simpan,
                child: const Text('Simpan')),
          ],
        ),
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
final Catatan? catatan	Parameter opsional. null berarti mode tambah, terisi berarti mode ubah.
late final TextEditingController _judul	Controller yang dibuat di initState() karena nilai awalnya bergantung pada widget.catatan.
TextEditingController(text: widget.catatan?.judul ?? '')	Mengisi kolom dengan data lama bila mode ubah, atau kosong bila mode tambah. Tanda ?. aman terhadap null.
widget.catatan	Cara State mengakses data milik widget induknya.
validator	Judul dan isi wajib diisi; bila kosong muncul pesan error.
maxLines: 6	Kolom isi catatan dibuat tinggi untuk beberapa baris teks.
_formKey.currentState!.validate()	Menjalankan validasi; bila gagal proses simpan dihentikan.
Catatan(id: widget.catatan?.id, ...)	Membuat objek dari isian. id tetap null saat catatan baru, dan memakai id lama saat ubah.
if (widget.catatan == null) tambah else ubah	Memilih DbHelper.tambah (INSERT) atau DbHelper.ubah (UPDATE).
Navigator.pop(context)	Menutup form dan kembali ke daftar, yang kemudian memuat ulang data.
baru ? 'Catatan Baru' : 'Ubah Catatan'	Judul AppBar menyesuaikan mode form.

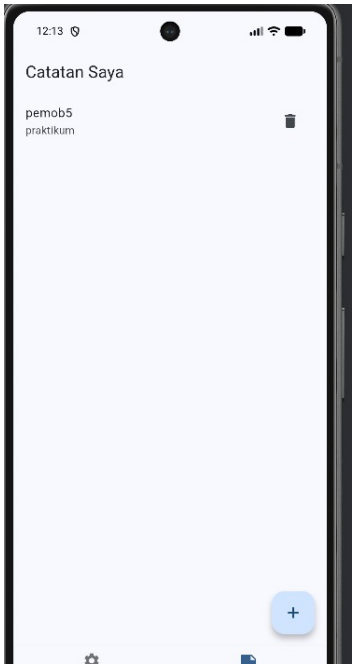
Hasil Praktikum Bagian A–D



Gambar 1. Pengaturan: nama, mode gelap, dan hitungan (Bagian A)



Gambar 2. Form Ubah Catatan (Bagian D)



Gambar 3. Daftar Catatan Saya (Bagian C)

Gambar 1 menampilkan halaman Pengaturan dengan nama "dika" dan mode gelap aktif yang tersimpan lewat shared_preferences; hitungan menunjukkan aplikasi sudah dibuka ke-7 kali, yang membuktikan data bertahan setelah aplikasi ditutup dan dibuka kembali. Navigasi bawah memuat dua tab: Pengaturan (A) dan Catatan SQLite (B–D). Gambar 2 adalah form yang sama dipakai untuk mengubah catatan "pemob5" dengan isi "praktikum" (judul AppBar "Ubah Catatan"). Gambar 3 menampilkan daftar yang membaca catatan dari database SQLite beserta ikon hapus dan tombol + untuk menambah catatan.

5. Latihan Mandiri

Latihan mandiri meliputi: (1) AlertDialog konfirmasi sebelum hapus, (2) kolom pencarian dengan where 'judul LIKE ?', (3) kolom dibuat dengan version 2 dan onUpgrade, serta (4) penyimpanan pilihan urutan dengan shared_preferences yang diterapkan pada orderBy.

Latihan 1: Dialog konfirmasi hapus

```
// Latihan 1: dialog konfirmasi sebelum menghapus
Future<void> _hapus(Catatan c) async {
  final yakin = await showDialog<bool>(
    context: context,
    builder: (ctx) => AlertDialog(
      title: const Text('Hapus catatan ini?'),
      content: Text('"${c.judul}" akan dihapus permanen.'),
      actions: [
```

```
        TextButton(
          onPressed: () => Navigator.pop(ctx, false),
          child: const Text('Batal'),
        ),
        TextButton(
          onPressed: () => Navigator.pop(ctx, true),
          child: const Text('Hapus'),
        ),
      ],
    ),
  );
  if (yakin == true) {
    await DbHelper.hapus(c.id!);
    _muat();
  }
}
```

Latihan 2 dan 4: Pencarian dan urutan

```
// Latihan 2 dan 4: pencarian dan urutan di DbHelper
static Future<List<Catatan>> semua(
    {String kata = '', bool terbaru = true}) async {
  final db = await database;
  final rows = await db.query(
    'catatan',
    where: kata.isEmpty ? null : 'judul LIKE ?',
    whereArgs: kata.isEmpty ? null : ['%$kata%'],
    orderBy: terbaru ? 'id DESC' : 'id ASC',
  );
  return rows.map(Catatan.fromMap).toList();
}

// di _CatatanPageState: kolom pencarian
TextField(
  controller: _cari,
  decoration: const InputDecoration(
    hintText: 'Cari judul catatan...',
    prefixIcon: Icon(Icons.search),
    border: OutlineInputBorder(),
  ),
  onChanged: (_) => _muat(),
),
```

```
// Latihan 4: simpan pilihan urutan dengan shared_preferences
bool _terbaru = true;

Future<void> _muatUrutan() async {
  final prefs = await SharedPreferences.getInstance();
  if (!mounted) return;
  setState(() => _terbaru = prefs.getBool('terbaru') ?? true);
  _muat();
}
```

```
Future<void> _ubahUrutan() async {
  final prefs = await SharedPreferences.getInstance();
  setState(() => _terbaru = !_terbaru);
  await prefs.setBool('terbaru', _terbaru);
  _muat();
}

// di AppBar
actions: [
  IconButton(
    icon: const Icon(Icons.sort),
    tooltip: _terbaru ? 'Terbaru dulu' : 'Terlama dulu',
    onPressed: _ubahUrutan,
  ),
],
```

Latihan 3: Kolom dibuat dan migrasi database

```
// Latihan 3: kolom dibuat dan migrasi versi 2
_db = await openDatabase(
  path,
  version: 2,                        // dinaikkan dari 1
  onCreate: (db, version) {
    return db.execute(
      'CREATE TABLE catatan('
      'id INTEGER PRIMARY KEY AUTOINCREMENT, '
      'judul TEXT NOT NULL, '
      'isi TEXT NOT NULL, '
      'dibuat TEXT)',
    );
  },
  onUpgrade: (db, oldVersion, newVersion) async {
    if (oldVersion < 2) {
      await db.execute(
          'ALTER TABLE catatan ADD COLUMN dibuat TEXT');
    }
  },
);

// di model Catatan: properti baru
final String? dibuat;

// saat membuat catatan baru di FormCatatanPage
final c = Catatan(
  id: widget.catatan?.id,
  judul: _judul.text.trim(),
  isi: _isi.text.trim(),
  dibuat: widget.catatan?.dibuat ??
      DateTime.now().toString().substring(0, 19),
);

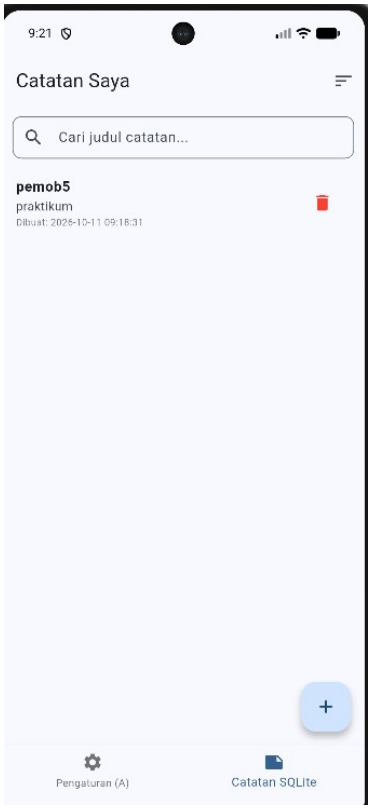
// di ListTile
```

```
subtitle: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text(c.isi, maxLines: 2,
        overflow: TextOverflow.ellipsis),
    Text('Dibuat: ${c.dibuat ?? '-'}',
        style: const TextStyle(fontSize: 11)),
  ],
),
```

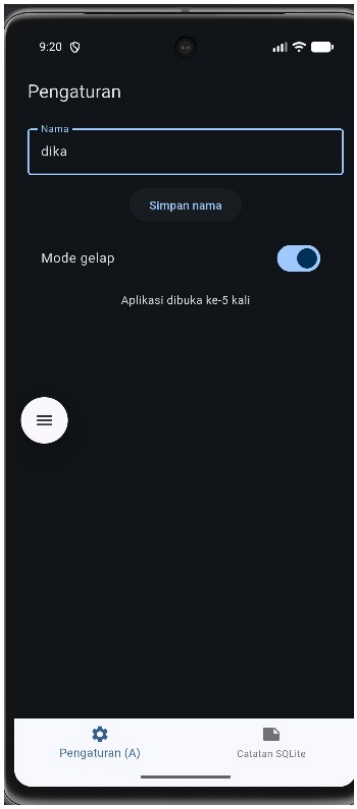
Bagian Kode	Penjelasan
showDialog<bool>(...)	Menampilkan dialog dan menunggu jawaban pengguna. Hasilnya bertipe bool: true untuk Hapus, false untuk Batal.
AlertDialog	Dialog konfirmasi berisi judul, isi, dan tombol aksi (actions).
Navigator.pop(ctx, true/false)	Menutup dialog sambil mengirim jawaban sebagai nilai kembalian.
if (yakin == true)	Data baru dihapus bila pengguna menjawab Hapus. Dibandingkan dengan true karena hasil dialog bisa null (dialog ditutup di luar tombol).
kata.isEmpty ? null : 'judul LIKE ?'	Bila kolom pencarian kosong, tanpa filter. Bila terisi, SQL memfilter judul yang cocok.
whereArgs: ['%$kata%']	Tanda % adalah wildcard LIKE: judul yang mengandung kata di posisi mana pun. Nilai dikirim lewat placeholder sehingga aman dari SQL injection.
onChanged: (_) => _muat()	Daftar disaring ulang setiap kali teks pencarian berubah.
orderBy: terbaru ? 'id DESC' : 'id ASC'	Mengubah urutan: DESC dari terbaru, ASC dari terlama.
prefs.getBool('terbaru') ?? true	Membaca pilihan urutan yang tersimpan; bawaannya terbaru dulu.
_ubahUrutan()	Membalik pilihan, menyimpannya ke shared_preferences, lalu memuat ulang daftar. Pilihan tetap diingat saat aplikasi dibuka lagi.
version: 2	Menaikkan versi database dari 1 ke 2 sebagai tanda skema berubah.
onUpgrade: (db, oldVersion, newVersion)	Dijalankan hanya bila versi yang diminta lebih besar dari versi database yang sudah ada di perangkat.
ALTER TABLE catatan ADD COLUMN dibuat TEXT	Menambah kolom baru pada tabel tanpa menghapus data lama.
onCreate (diperbarui)	Tabel baru yang dibuat dari nol langsung memiliki kolom dibuat.

Bagian Kode	Penjelasan
DateTime.now().toString().substring(0, 19)	Waktu sekarang dipotong menjadi format "2026-10-11 09:18:31".
widget.catatan?.dibuat ?? ...	Saat mengubah catatan, tanggal dibuat yang lama dipertahankan; hanya catatan baru yang diberi waktu sekarang.

Pengamatan bila version tidak dinaikkan: onUpgrade tidak dijalankan karena versi database di perangkat sama dengan versi yang diminta. Akibatnya kolom dibuat tidak pernah ditambahkan, dan operasi yang memakai kolom tersebut gagal dengan galat "no such column". Pada masa pengembangan, solusinya menaikkan version atau menghapus data aplikasi (uninstall/Clear data) lalu menjalankan ulang.



Gambar 4. Daftar dengan kolom pencarian, tombol urutan, dan tanggal dibuat



Gambar 5. Halaman Pengaturan setelah aplikasi dibuka kembali

Pada Gambar 4 halaman Catatan Saya kini memiliki kolom pencarian "Cari judul catatan...", ikon urutan di AppBar, judul catatan tebal, dan keterangan "Dibuat: 2026-10-11 09:18:31". Ikon hapus berwarna merah. Gambar 5 menampilkan halaman Pengaturan dengan nama, mode gelap, dan hitungan pembukaan yang dimuat dari penyimpanan.

6. Tugas: Aplikasi Pencatat Pengeluaran

Aplikasi Pencatat Pengeluaran memakai tabel pengeluaran (id, nama, jumlah bertipe integer, kategori, dan tanggal) dengan CRUD lengkap: daftar, tambah, ubah, dan hapus. Form memiliki validasi (nama wajib, jumlah angka lebih dari 0, kategori dari dropdown). Halaman utama menampilkan total pengeluaran, dan satu pengaturan (mode gelap) disimpan dengan shared_preferences. Data tetap ada setelah aplikasi ditutup dan dibuka kembali.

Model dan akses data (SQLite)

```
// ===== MODEL dan AKSES DATA =====
class Pengeluaran {
  final int? id;
  final String nama;
  final int jumlah;
  final String kategori;
  final String tanggal;

  const Pengeluaran({
    this.id,
    required this.nama,
    required this.jumlah,
    required this.kategori,
    required this.tanggal,
  });

  Map<String, Object?> toMap() => {
        'id': id,
        'nama': nama,
        'jumlah': jumlah,
        'kategori': kategori,
        'tanggal': tanggal,
      };

  factory Pengeluaran.fromMap(Map<String, Object?> m) =>
      Pengeluaran(
        id: m['id'] as int,
        nama: m['nama'] as String,
        jumlah: m['jumlah'] as int,
        kategori: m['kategori'] as String,
        tanggal: m['tanggal'] as String,
      );
}

class DbPengeluaran {
  static Database? _db;

  static Future<Database> get database async {
    if (_db != null) return _db!;
    final path =
        p.join(await getDatabasesPath(), 'pengeluaran.db');
    _db = await openDatabase(
      path,
```

```
      version: 1,
      onCreate: (db, version) {
        return db.execute(
          'CREATE TABLE pengeluaran('
          'id INTEGER PRIMARY KEY AUTOINCREMENT, '
          'nama TEXT NOT NULL, '
          'jumlah INTEGER NOT NULL, '
          'kategori TEXT NOT NULL, '
          'tanggal TEXT NOT NULL)',
        );
      },
    );
    return _db!;
  }

  static Future<int> tambah(Pengeluaran x) async {
    final db = await database;
    return db.insert('pengeluaran', x.toMap());
  }

  static Future<List<Pengeluaran>> semua() async {
    final db = await database;
    final rows =
        await db.query('pengeluaran', orderBy: 'id DESC');
    return rows.map(Pengeluaran.fromMap).toList();
  }

  static Future<int> ubah(Pengeluaran x) async {
    final db = await database;
    return db.update('pengeluaran', x.toMap(),
        where: 'id = ?', whereArgs: [x.id]);
  }

  static Future<int> hapus(int id) async {
    final db = await database;
    return db.delete('pengeluaran',
        where: 'id = ?', whereArgs: [id]);
  }
}
```

Aplikasi dan pengaturan tema (shared_preferences)

```
// ===== APLIKASI + PENGATURAN TEMA (shared_preferences) =====
class PencatatApp extends StatefulWidget {
  const PencatatApp({super.key});

  @override
  State<PencatatApp> createState() => _PencatatAppState();
}

class _PencatatAppState extends State<PencatatApp> {
  bool _gelap = false;
```

```
  @override
  void initState() {
    super.initState();
    _muatTema();
  }

  Future<void> _muatTema() async {
    final prefs = await SharedPreferences.getInstance();
    if (!mounted) return;
    setState(() => _gelap = prefs.getBool('gelap') ?? false);
  }

  Future<void> _gantiTema(bool nilai) async {
    setState(() => _gelap = nilai);
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool('gelap', nilai);
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Pencatat Pengeluaran',
      theme: ThemeData(
        colorSchemeSeed: Colors.teal,
        brightness: _gelap ? Brightness.dark : Brightness.light,
        useMaterial3: true,
      ),
      home: PengeluaranPage(gelap: _gelap, onGantiTema: _gantiTema),
    );
  }
}
```

Halaman utama: total dan daftar

```
// ===== HALAMAN UTAMA: total + daftar =====
class _PengeluaranPageState extends State<PengeluaranPage> {
  late Future<List<Pengeluaran>> _future;

  @override
  void initState() {
    super.initState();
    _future = DbPengeluaran.semua();
  }

  void _muat() {
    setState(() => _future = DbPengeluaran.semua());
  }

  Future<void> _buka([Pengeluaran? x]) async {
    await Navigator.push(
      context,
      MaterialPageRoute(
          builder: (_) => FormPengeluaranPage(pengeluaran: x)),
```

```
    );
    if (!mounted) return;
    _muat();
  }

  IconData _ikon(String kategori) {
    switch (kategori) {
      case 'Makanan':
        return Icons.restaurant;
      case 'Transportasi':
        return Icons.directions_car;
      case 'Hiburan':
        return Icons.movie;
      case 'Belanja':
        return Icons.shopping_bag;
      default:
        return Icons.category;
    }
  }

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;
    return Scaffold(
      appBar: AppBar(
        title: const Text('Pencatat Pengeluaran'),
        actions: [
          const Icon(Icons.dark_mode, size: 18),
          Switch(value: widget.gelap,
              onChanged: widget.onGantiTema),
        ],
      ),
      body: FutureBuilder<List<Pengeluaran>>(
        future: _future,
        builder: (context, snapshot) {
          if (snapshot.connectionState ==
              ConnectionState.waiting) {
            return const Center(
                child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return Center(
                child: Text('Galat: ${snapshot.error}'));
          }
          final data = snapshot.data!;
          final total =
              data.fold<int>(0, (s, e) => s + e.jumlah);
          return Column(
            children: [
              Container(
                width: double.infinity,
                margin: const EdgeInsets.all(16),
                padding: const EdgeInsets.all(16),
                decoration: BoxDecoration(
```

```
                color: warna.primaryContainer,
                borderRadius: BorderRadius.circular(16),
              ),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  const Text('Total Pengeluaran'),
                  Text('Rp $total',
                      style: const TextStyle(
                          fontSize: 28,
                          fontWeight: FontWeight.bold)),
                ],
              ),
            ),
            Expanded(
              child: data.isEmpty
                  ? const Center(
                      child: Text('Belum ada data pengeluaran.'))
                  : ListView.builder(
                      itemCount: data.length,
                      itemBuilder: (context, i) {
                        final x = data[i];
                        return ListTile(
                          leading: CircleAvatar(
                              child: Icon(_ikon(x.kategori))),
                          title: Text(x.nama,
                              style: const TextStyle(
                                  fontWeight:
                                      FontWeight.bold)),
                          subtitle: Text(
                              '${x.kategori} • ${x.tanggal}'),
                          onTap: () => _buka(x),
                          trailing: Row(
                            mainAxisSize: MainAxisSize.min,
                            children: [
                              Text('Rp ${x.jumlah}',
                                  style: const TextStyle(
                                      color: Colors.red,
                                      fontWeight:
                                          FontWeight.bold)),
                              IconButton(
                                icon: const Icon(Icons.delete),
                                onPressed: () async {
                                  await DbPengeluaran
                                      .hapus(x.id!);
                                  _muat();
                                },
                              ),
                            ],
                          ),
                        );
                      },
                    ),
            ),
```

```
            ],
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => _buka(),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Form tambah dan ubah dengan validasi

```
// ===== FORM TAMBAH / UBAH dengan validasi =====
class _FormPengeluaranPageState
    extends State<FormPengeluaranPage> {
  static const _kategori = [
    'Makanan', 'Transportasi', 'Hiburan', 'Belanja', 'Lainnya'
  ];

  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _nama;
  late final TextEditingController _jumlah;
  String? _pilihan;

  @override
  void initState() {
    super.initState();
    final x = widget.pengeluaran;
    _nama = TextEditingController(text: x?.nama ?? '');
    _jumlah = TextEditingController(
        text: x == null ? '' : '${x.jumlah}');
    _pilihan = x?.kategori;
  }

  @override
  void dispose() {
    _nama.dispose();
    _jumlah.dispose();
    super.dispose();
  }

  Future<void> _simpan() async {
    if (!_formKey.currentState!.validate()) return;
    final x = Pengeluaran(
      id: widget.pengeluaran?.id,
      nama: _nama.text.trim(),
      jumlah: int.parse(_jumlah.text),
      kategori: _pilihan!,
      tanggal: widget.pengeluaran?.tanggal ??
          DateTime.now().toString().substring(0, 10),
    );
```

```
    if (widget.pengeluaran == null) {
      await DbPengeluaran.tambah(x);
    } else {
      await DbPengeluaran.ubah(x);
    }
    if (!mounted) return;
    Navigator.pop(context);
  }

  @override
  Widget build(BuildContext context) {
    final baru = widget.pengeluaran == null;
    return Scaffold(
      appBar: AppBar(title: Text(
          baru ? 'Tambah Pengeluaran' : 'Ubah Pengeluaran')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _nama,
              decoration: const InputDecoration(
                labelText: 'Nama Pengeluaran',
                border: OutlineInputBorder(),
              ),
              validator: (v) => (v == null || v.trim().isEmpty)
                  ? 'Nama wajib diisi' : null,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _jumlah,
              keyboardType: TextInputType.number,
              decoration: const InputDecoration(
                labelText: 'Jumlah (Rp)',
                border: OutlineInputBorder(),
              ),
              validator: (v) {
                final n = int.tryParse(v ?? '');
                if (n == null || n <= 0) {
                  return 'Jumlah harus angka lebih dari 0';
                }
                return null;
              },
            ),
            const SizedBox(height: 12),
            DropdownButtonFormField<String>(
              value: _pilihan,
              decoration: const InputDecoration(
                labelText: 'Kategori',
                border: OutlineInputBorder(),
              ),
              items: _kategori
                  .map((k) => DropdownMenuItem(
```

```
                      value: k, child: Text(k)))
                  .toList(),
              onChanged: (v) => setState(() => _pilihan = v),
              validator: (v) =>
                  v == null ? 'Pilih kategori' : null,
            ),
            const SizedBox(height: 24),
            ElevatedButton(onPressed: _simpan,
                child: const Text('Simpan')),
          ],
        ),
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
class Pengeluaran	Model satu pengeluaran dengan lima kolom: id, nama, jumlah (int), kategori, dan tanggal.
jumlah INTEGER NOT NULL	Jumlah disimpan sebagai bilangan bulat sehingga bisa dijumlahkan.
DbPengeluaran	Kelas akses data berisi tambah, semua, ubah, dan hapus; dipisah dari UI agar kode rapi.
PencatatApp (StatefulWidget)	Menyimpan pilihan tema di level aplikasi sehingga seluruh aplikasi (bukan hanya satu halaman) berganti terang atau gelap.
_muatTema() dan _gantiTema()	Membaca dan menyimpan nilai gelap di shared_preferences, sehingga pilihan tema tetap ada setelah restart.
brightness: _gelap ? dark : light	Mengubah tema aplikasi sesuai pilihan; colorSchemeSeed: Colors.teal memberi warna dasar hijau kebiruan.
onGantiTema: _gantiTema	Fungsi callback dikirim ke halaman utama agar saklar di AppBar dapat mengubah tema.
Switch + Icons.dark_mode	Saklar mode gelap di AppBar dengan ikon bulan.
data.fold<int>(0, (s, e) => s + e.jumlah)	Menjumlahkan seluruh jumlah pengeluaran di Dart: mulai dari 0, lalu menambah jumlah setiap item. Hasilnya total pengeluaran. Alternatifnya memakai SELECT SUM(jumlah).
Container + primaryContainer	Kartu total dengan warna latar dari tema dan sudut membulat.
Expanded(child: ListView...)	Daftar mengisi sisa ruang di bawah kartu total; tanpa Expanded, ListView di dalam Column menimbulkan galat tinggi tak terbatas.

Bagian Kode	Penjelasan
data.isEmpty ? Text(...) : ListView	Bila belum ada data tampil "Belum ada data pengeluaran."
_ikon(String kategori)	Fungsi pembantu yang memilih ikon sesuai kategori memakai switch.
trailing: Row(mainAxisSize: MainAxisSize.min)	Menaruh jumlah (merah) dan ikon hapus berdampingan di sisi kanan; ukuran Row dibatasi seperlunya.
onTap: () => _buka(x)	Mengetuk item membuka form dalam mode ubah (Update).
FormPengeluaranPage(pengeluaran: x)	Satu form untuk tambah dan ubah, sama seperti pada Bagian D.
int.tryParse(v ?? '')	Mengubah teks menjadi angka; null bila bukan angka. Validasi menolak null atau nilai ≤ 0.
DropdownButtonFormField + validator	Dropdown kategori (Makanan, Transportasi, Hiburan, Belanja, Lainnya); wajib dipilih.
DateTime.now().toString().substring(0, 10)	Tanggal otomatis hari ini dalam format "2026-10-11".
widget.pengeluaran?.tanggal ?? ...	Saat mengubah data, tanggal lama dipertahankan.



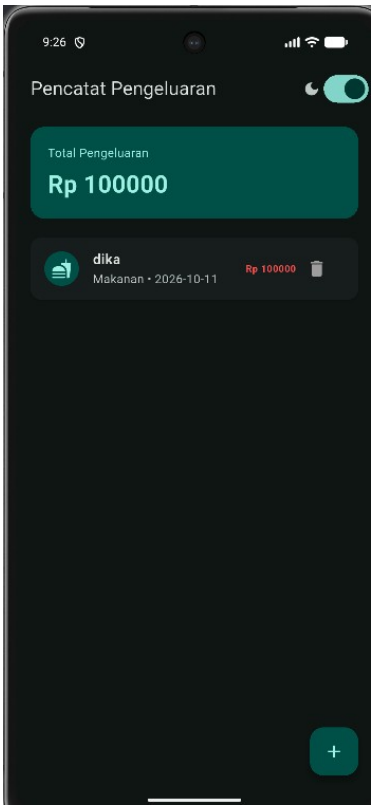
Gambar 6. Halaman utama (tema terang), total Rp 0, data kosong



Gambar 7. Mode gelap diaktifkan lewat saklar di AppBar



Gambar 8. Form Tambah Pengeluaran, dropdown kategori terbuka



Gambar 9. Daftar setelah data ditambah, total Rp 100000

Gambar 6 menampilkan halaman utama saat belum ada data: kartu "Total Pengeluaran Rp 0", teks "Belum ada data pengeluaran.", tombol +, dan saklar mode gelap di AppBar. Gambar 7 menunjukkan seluruh aplikasi berganti ke tema gelap setelah saklar diaktifkan. Gambar 8 adalah form Tambah Pengeluaran berisi kolom Nama Pengeluaran, Jumlah (Rp), dan dropdown kategori dengan lima pilihan. Gambar 9 menampilkan pengeluaran "dika" (Makanan, 2026-10-11, Rp 100000) dengan ikon kategori dan ikon hapus; total pada kartu ikut berubah menjadi Rp 100000.

7. Jawaban Pertanyaan Refleksi

1. Mengapa shared_preferences kurang cocok untuk menyimpan ratusan catatan? Apa pilihan yang lebih baik?

shared_preferences dirancang untuk pasangan key-value kecil seperti pengaturan. Ia tidak mendukung pencarian, penyaringan, pengurutan, maupun hubungan antar data, sehingga untuk ratusan catatan semua data harus dibaca dan diolah manual di Dart, dan performanya menurun seiring bertambahnya data. Pilihan yang lebih baik adalah database SQLite (paket sqflite) yang menyimpan data terstruktur dalam tabel dan dapat di-query dengan WHERE, ORDER BY, maupun LIKE secara efisien.

2. Mengapa query memakai placeholder ? dan whereArgs, bukan menyambung teks langsung?

Karena menyambung teks dari pengguna langsung ke perintah SQL membuka celah SQL injection. Pengguna bisa memasukkan teks yang berisi perintah SQL (misalnya menutup tanda kutip lalu menambahkan perintah lain) dan perintah itu ikut dijalankan. Dengan placeholder ?, nilai di whereArgs dikirim terpisah dan selalu diperlakukan sebagai data, bukan sebagai bagian perintah, sehingga aman. Cara ini juga menangani karakter khusus seperti tanda kutip dengan benar.

3. Mengapa setelah kembali dari halaman form, daftar perlu dimuat ulang? Bagaimana caranya pada kode ini?

Karena daftar menampilkan hasil Future yang sudah dibuat sebelumnya (_future). FutureBuilder tidak mengambil data lagi dengan sendirinya, sehingga setelah data di database berubah, layar masih menampilkan hasil query lama. Pada kode ini, _buka() memakai await Navigator.push(...) untuk menunggu form ditutup, memeriksa if (!mounted) return, lalu memanggil _muat() yang membuat Future baru lewat setState sehingga FutureBuilder memuat ulang data. Hal yang sama dilakukan setelah menghapus catatan.

4. Apa fungsi onCreate dan onUpgrade, dan kapan masing-masing dijalankan?

onCreate dipakai untuk membuat struktur database (tabel) dan hanya dijalankan sekali, yaitu saat file database belum ada dan dibuat pertama kali. onUpgrade dipakai untuk memperbarui skema (misalnya ALTER TABLE ADD COLUMN) tanpa menghapus data lama, dan dijalankan ketika version yang diberikan ke openDatabase lebih besar daripada version database yang sudah ada di perangkat. Bila version tidak dinaikkan, onUpgrade tidak dijalankan sehingga perubahan skema tidak diterapkan.

8. Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa data di memori akan hilang saat aplikasi ditutup, sehingga diperlukan penyimpanan lokal. shared_preferences cocok untuk pengaturan kecil seperti nama, mode gelap, dan hitungan pembukaan, sedangkan SQLite lewat sqflite cocok untuk data terstruktur dan banyak seperti catatan dan pengeluaran. Operasi CRUD dipetakan ke insert, query, update, dan delete, dengan placeholder ? dan whereArgs untuk menghindari SQL injection. Daftar perlu dimuat ulang setelah data berubah, dan perubahan skema dikelola dengan version dan onUpgrade. Seluruh konsep ini diterapkan pada aplikasi Catatan dan Pencatat Pengeluaran yang datanya tetap ada setelah aplikasi dibuka kembali.